NFL Predictor v1: Project Plan and Report

Oct 29, 2026 · @James Havrilla

Author: James Havrilla III

1. Framing the problem

The goal of v1 is to pick the straight-up winner of every NFL game before kickoff, using only team and game data. I deliberately left betting lines out. A model that learns from the point spread mostly repeats the market, and I want this baseline to show what on-field data can do by itself.

I followed the project checklist in Aurélien Géron's *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (Chapter 2 and Appendix B). The sections below follow its steps.

Decision	Choice
Objective	Pick the winner of each game. The model outputs a home-win probability and picks the side above 50%.
Problem type	Supervised binary classification (home team wins = 1), retrained in batch each week
Primary metric	Accuracy of picks
Secondary metrics	Log loss and Brier score, which show whether the probabilities can be trusted
Minimum bar	Beat "always pick the home team" and a plain Elo rating on the holdout seasons
Target	63 to 65% holdout accuracy, in line with an earlier prototype of mine (64.3% without lines)
Excluded	Point spreads, totals, moneylines and anything else derived from betting markets
Ties	Removed from training and scoring (roughly one per season)

For comparison I used three reference points: always picking the home team, a FiveThirtyEight-style Elo rating, and my earlier prototype, which combined Elo with rolling EPA.

Assumptions I set out to check:

- Team strength carries over from week to week, and partly from season to season.
- Home-field advantage changes over time and has shrunk since about 2016.
- Every feature can be known before kickoff, so nothing leaks from the game itself.
- A change at quarterback moves a team's strength more than any other single factor.
- The move to a 17-game schedule in 2021 does not change the patterns the model relies on.

2. Getting the data

The dataset covers 7,548 scheduled games from 1999 through 2026, of which 7,383 had been played as of week 7 of 2026. All of it comes from nflverse's public data releases on GitHub. The loader drops all eight betting columns as soon as the schedule is read and checks that none remain.

Source	Contents	File
nflverse schedules	Teams, dates, scores, rest days, division flag, site and starting quarterbacks	games.csv
nflverse weekly team stats	Box score and EPA for each team in every game, 1999 to 2026	stats_team_week_YYYY.csv

I split the data by season before exploring anything:

Set	Seasons	Use
Training	1999 to 2022	Exploration, cross-validation and tuning
Holdout	2023 to 2025 (about 850 games)	Evaluated once, at the end
Live	2026	Weekly predictions and monitoring

Splitting by season instead of at random matters here. A random split would let the model train on games played after the ones it is tested on.

Cleaning steps:

- Removed the betting columns: moneylines, spread, spread odds, total and over/under odds.
- Merged relocated franchises so their ratings carry over (Oakland to Las Vegas, San Diego to the Los Angeles Chargers, St. Louis to the Los Angeles Rams).
- Dropped duplicate team-stat rows. About 0.3% of games are missing team stats, and the pipeline fills those gaps with the median.

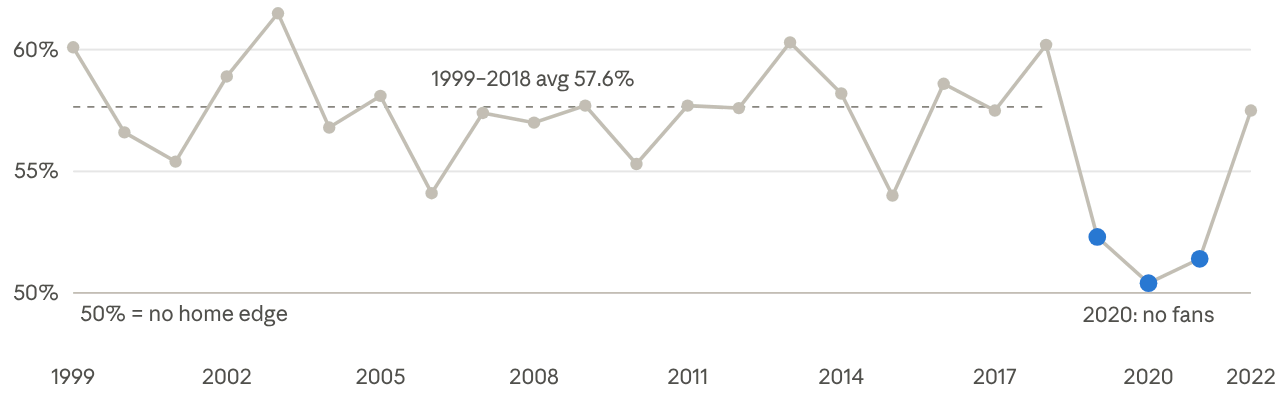
One disclosure: while checking the files I printed the home-team win rate for the holdout seasons (54.6%). That is a baseline figure rather than a modeling choice, and I did not look at anything else in the holdout.

3. Exploring the data

A plain Elo rating already picks 64.5% of winners in the training seasons, and home-field advantage turned out to be the least stable signal in the data.

Home teams won less often from 2019 to 2021

Home win rate by season, non-neutral games. 1999–2018 average 57.6%; 2019–2021 average 51.4%



nflverse schedules · training seasons 1999–2022 · ties and neutral-site games excluded

What I found in the 6,407 training games (ties excluded):

- About 23% of games end within 3 points and 47% within 7. Many outcomes are close to a coin flip, which puts a ceiling on any model's accuracy.

- The strongest single signals are Elo difference (correlation of 0.32 with a home win), points-scored difference (0.28), offensive EPA per play (0.26) and passing EPA per dropback (0.25).
- Defensive measures add less: points allowed (−0.20), sack rate (−0.15) and passing EPA allowed (−0.14).
- Quarterback changes matter a lot. Home teams starting a new quarterback won 44.9% of the time, compared with 58.2% otherwise.
- Early-season form is noisy. Elo picks 62.4% of games in a team's first three weeks and 65.8% after week 10.
- Over 2001 to 2022, always picking the home team is right 56.5% of the time and default Elo 64.5%. Those are the baselines to beat.

4. Preparing the features

Each game is described by 20 home-minus-away features, all built from games played before it. One scikit-learn transformer produces them, and I exposed its settings as hyperparameters so the search in step 6 could tune them along with the model.

Group	Features	Tunable settings
Elo rating	Rating difference and Elo win probability, including the home bump	K-factor, home bump (in Elo points), reversion to the mean at the start of each season
Rolling form	Points scored and allowed, offensive and defensive EPA per play, passing EPA per dropback (offense and defense), rushing EPA per carry, sack rate (offense and defense), turnover margin, penalty yards	Half-life of the weighted average, in games
Context	Rest-day difference, division game, neutral site, playoff game, new starting quarterback (home and away), games played so far this season	On/off switch for each feature group

To prevent leakage:

- Elo ratings update after each game, so a game only sees ratings from before kickoff.

- Rolling form is an exponentially weighted average shifted back one game and carried across seasons.
- Weather, injury reports and anything else known only close to kickoff are left out of v1.
- The starting quarterback comes from the schedule. This assumes the starter is known before kickoff, which holds for most games but not for surprise starts.

To test for leakage, I changed the result of one 2015 game and rebuilt the features. No feature changed for that game or any earlier game; only later games were affected.

After the transformer, the pipeline fills missing values with the median and standardizes features for the models that need scaling.

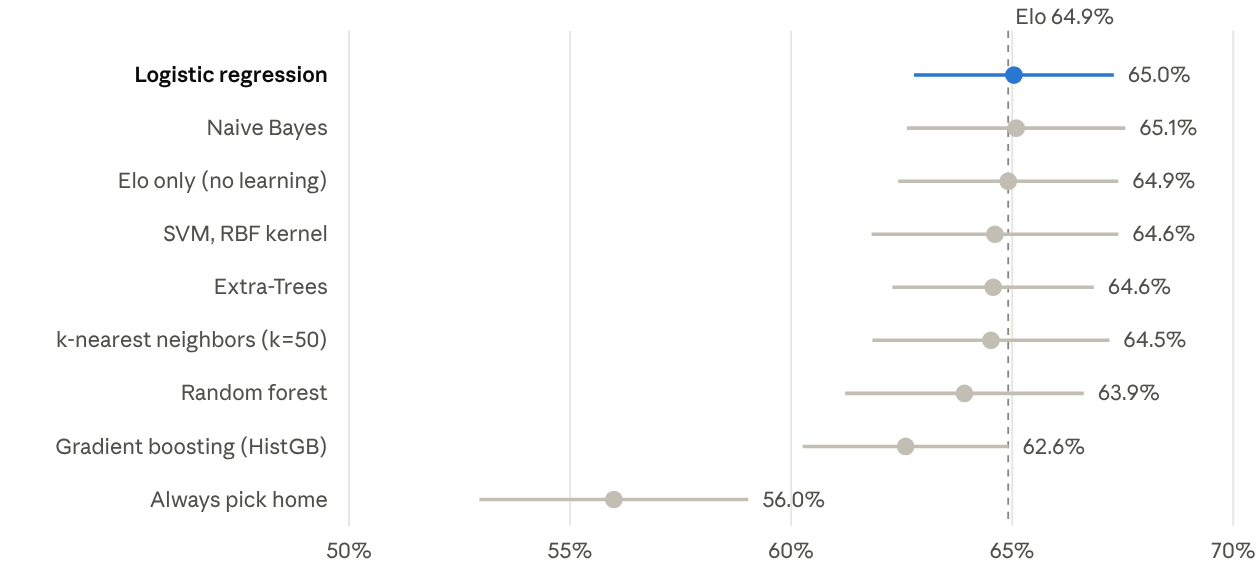
5. Shortlisting models

With default settings, logistic regression scored best (65.0% accuracy, log loss 0.625). Even so, every learned model finished within normal season-to-season variation of a plain Elo rating (64.9%).

I validated with walk-forward cross-validation by season. Each fold trains on every season from 2001 up to the prior year and tests on the next season, covering 2011 through 2022 (12 folds). The 1999 and 2000 seasons are used only to warm up the ratings. Géron's examples use shuffled folds, which would leak future information for game data.

Learned models barely beat plain Elo

Walk-forward accuracy, 12 test seasons (2011–2022). Dot = mean; whisker = ±1 std across seasons



results/experiments.csv · stage = shortlist · 5,889 games, 2001–2022

Model	Log loss	Brier	Comment
Logistic regression	0.625	0.218	Best on all three metrics
Elo only	0.628	0.219	No learning, used as the bar to beat
KNN (k=50)	0.630	0.220	
Extra-Trees	0.632	0.221	
Random forest	0.634	0.222	
SVM, RBF kernel	0.641	0.225	Slowest (28 s), and its probabilities need calibration
HistGB	0.659	0.232	Overfits with default settings
Always home	0.686	0.247	
Naive Bayes	1.252	0.274	Good picks, but badly overconfident probabilities

I carried logistic regression, gradient boosting and Extra-Trees into tuning. They come from three different model families and make different kinds of errors, which also makes them reasonable candidates for an ensemble. I dropped Naive Bayes and the SVM because their probabilities were unreliable.

For tuning I select on log loss and record accuracy alongside it. Accuracy moves in steps and swings about two points from season to season, so selecting on it would mostly mean chasing noise.

6. Tuning and the holdout result

The final model is a soft-voting ensemble of the three tuned models. On the 2023 to 2025 holdout it picked 63.2% of games correctly (95% CI 60.0% to 66.5%), which is statistically tied with plain Elo (64.1%) and well ahead of always picking the home team (54.9%).

I tuned each shortlisted model with a randomized search (60 draws for logistic regression, 40 for gradient boosting, 20 for Extra-Trees). The search covered model settings and the feature settings together: Elo K-factor, home bump, season reversion, form half-life, and whether to include the form and context groups.

Model (tuned)	CV accuracy	CV log loss	Notes
Soft-voting ensemble	65.5%	0.6245	Selected
Logistic regression	65.4%	0.6250	C = 0.018, Elo K = 32.7, home bump = 46, form half-life = 7.7 games
Extra-Trees	64.1%	0.6267	Short form half-life (2.4 games)
Gradient boosting	64.7%	0.6301	The search turned off the context features

Before running the holdout, I set the selection rule: the candidate with the lowest walk-forward log loss would be retrained on 2001 to 2022 and scored once, with no changes afterward. That candidate was the ensemble.

Holdout, 2023 to 2025 (854 games)	Accuracy	95% CI	Log loss	Brier
v1 ensemble	63.2%	60.0% to 66.5%	0.629	0.220
Plain Elo	64.1%	60.8% to 67.3%	0.635	0.222
Always home	54.9%	51.6% to 58.3%	0.689	0.248

The ensemble and Elo disagreed on only 71 of the 854 games. Elo was right on 39 of them and the ensemble on 32. The accuracy gap is −0.8 points with a bootstrap 95% interval of −2.7 to +1.2, so it is not meaningful. The ensemble's probabilities were somewhat better (log loss 0.629 against 0.635, interval −0.013 to 0.000).

By season, the ensemble scored 60.7% in 2023, 66.7% in 2024 and 62.3% in 2025. Its confidence was meaningful: games it rated above 80% went its way 87.7% of the time (65 games), while games rated 50 to 60% were a coin flip (50.6% of 332).

On the training seasons the ensemble was well calibrated. Across ten probability bins, predicted and observed home-win rates stayed within about 3 points of each other in nine bins, the exception being the 46% bin (observed 36%).

7. Results and lessons

v1 is a sound, leak-free baseline that picks about 63 to 65% of NFL games without any betting data. It did not beat a well-built Elo rating on picks, and that result shapes what I do next.

What worked:

- Walk-forward validation gave an honest read. The cross-validation estimate (65.5%) and the holdout result (63.2%) are within normal season-to-season variation of each other.
- Fixing the selection rule before touching the holdout kept the final number clean.
- Tuning the feature settings along with the models paid off more than the model choice did. Every tuned model raised the Elo K-factor from the usual 20 to between 23 and 35, meaning ratings should react faster to recent games than the FiveThirtyEight default.
- The probabilities are trustworthy. Calibration was close on the training seasons, and the model's most confident picks were right 88% of the time on the holdout.

What did not work:

- More complex models added almost nothing over Elo. With mostly team-level features, nearly all of the signal is already captured by a rating system.
- Out of the box, gradient boosting overfit, which matches Géron's discussion of shrinkage and early stopping in Chapter 7. Naive Bayes picked well but was badly overconfident.
- The rolling EPA features are highly correlated with each other and with points scored, so the logistic model spread small, sometimes odd-signed weights across them.

Assumptions, revisited:

- Home-field advantage did shrink. The home team won 57.6% of games from 1999 to 2018 but only 51.4% from 2019 to 2021, and always picking the home team was right just 54.9% of the time in the holdout.
- Quarterback changes were the largest single context signal (a 13-point drop in home win rate).
- The 17-game schedule did not visibly change performance. The 2024 holdout season was the strongest of the three.

Limitations:

- Starting quarterbacks come from the schedule, which is only certain once the game is played. Surprise starts are a small source of leakage.

- There are no injury, weather or roster inputs beyond the quarterback.
- Results on 854 holdout games carry roughly a ±3-point margin of error, so small differences between models are not meaningful.

Next steps for v2:

- Rate quarterbacks individually (EPA per dropback with shrinkage) instead of flagging a change.
- Re-estimate home-field advantage over time instead of treating it as a single number.
- Reduce the overlapping EPA features with PCA or regularization (Géron, Chapter 8).
- Add injury reports through the nflverse injuries data, timestamped before kickoff.

8. Weekly use and monitoring

Each week I run one script, predict_week.py. It downloads the latest nflverse data, retrains the model on every completed game, writes that week's picks to a CSV and scores all earlier picks against final results.

In the 2026 season to date, the model picked 57.6% of 106 games (95% CI 48.0% to 67.1%). That is below the holdout result, but seven weeks is a small sample and early-season form is the noisiest part of the year. I will treat a full season below 60% as the signal to revisit the model.

Week 8 picks, ordered by confidence:

Away	Home	Pick	Confidence
TEN	CIN	CIN	85%
IND	JAX	JAX	82%
LAC	LA	LA	78%
BAL	BUF	BUF	73%
ARI	DAL	DAL	73%
NE	MIA	NE	69%
CLE	PIT	PIT	67%
CAR	GB	GB	67%
CHI	SEA	SEA	66%

Away	Home	Pick	Confidence
KC	DEN	DEN	61%
PHI	WAS	WAS	59%
LV	NYJ	NYJ	55%
MIN	DET	DET	54%
ATL	TB	TB	52%

9. Experiment log

I logged all 136 runs to results/experiments.csv, each with its settings and scores, so any result in this report can be traced back to the run that produced it.

Stage	Runs	What was compared
Shortlist	9	Default models and two baselines, walk-forward CV
Tuning	120	60 logistic, 40 gradient boosting and 20 Extra-Trees settings
Final selection	4	Tuned models and their ensemble, walk-forward CV
Holdout	3	Final model, Elo and always-home, scored once

References

- Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed., early release). O'Reilly Media.
- nflverse data releases, schedules and weekly team stats, accessed October 29, 2026.
- scikit-learn 1.9 documentation: TimeSeriesSplit, probability calibration and HistGradientBoostingClassifier.